DealFlow360 Implementation Specification

Next.js build spec for autonomous agent execution — 24 hour hackathon

Version 1.0

0. How to use this document

This is a build specification written to be executed by a coding agent. Read it end to end before writing any code.

Execution rules for the agent:

1. Follow the phases in order. Do not begin Phase 2 until Phase 1 verification passes.
2. `prisma/schema.prisma` is the single source of truth for data shape. Never invent a field name. If something is missing from the schema, stop and ask.
3. Every module talks to other modules through functions exported from `lib/services/*`. A module must never query another module's tables directly.
4. All monetary values are `Decimal(12,2)` in Prisma and `Prisma.Decimal` in TypeScript. Never use JavaScript `number` for money.
5. After each phase, run the stated verification command. If it fails, fix before continuing.

---

1. Technology stack

Layer	Choice	Notes
Framework	Next.js 15, App Router, TypeScript strict	One app, API routes and UI together
Database	PostgreSQL 16 (Docker)	Relational integrity + row locking required
ORM	Prisma 6	`$transaction` + `$queryRaw` for `SELECT ... FOR UPDATE`
Cache / queue backend	Redis 7 (Docker) via `ioredis`	Price list cache, rate limiter, BullMQ backend
Background jobs	BullMQ, **separate Node process**	Must not run inside the Next.js server
Auth	Self-built: `argon2` + `jose` JWT + httpOnly cookies	No Firebase, no NextAuth
Validation	Zod, shared between client and server	Lives in `lib/contracts/`
UI	Tailwind CSS + shadcn/ui	
Client data	TanStack Query	Mutations and polling only; reads prefer RSC
Charts	Recharts	Dashboard only
PDF (optional)	`@react-pdf/renderer`	Quote export, only if time permits

1.1 Next.js-specific constraints the agent must respect

These cause silent, hard-to-debug failures if ignored.

- **Prisma singleton.** Instantiate `PrismaClient` once in `lib/db.ts` and cache it on `globalThis` in development, otherwise hot reload exhausts the connection pool.
- **Middleware runs on the Edge runtime.** It may only verify JWTs using `jose`. It must never import `argon2`, `prisma`, or `ioredis`. Middleware does coarse route gating; fine-grained role checks happen inside route handlers.
- **Route handlers touching the database must opt into Node.** Add `export const runtime = "nodejs";` to every handler that imports Prisma, argon2, or ioredis.
- **BullMQ must not be started by Next.js.** It lives in `worker/index.ts`, run as its own process with `tsx watch`. Next.js only enqueues jobs.
- **Server Actions are permitted** for simple form mutations, but every endpoint listed in section 5 must exist as a real route handler, because the customer portal and load tests call them directly.
- **Do not cache authenticated reads.** Use `export const dynamic = "force-dynamic"` on dashboard and quote pages.

---

2. Prerequisites to install

Run these before Phase 0.

```
# Required tooling
node --version    # must be 20.x or 22.x LTS
docker --version  # Docker Desktop must be running
npm i -g pnpm tsx
```

No other global installs are needed. Everything else is a project dependency.

2.1 Project dependencies

```
pnpm add next@15 react react-dom @prisma/client ioredis bullmq \
  argon2 jose zod @tanstack/react-query recharts \
  clsx tailwind-merge class-variance-authority lucide-react

pnpm add -D prisma typescript @types/node @types/react \
  tailwindcss postcss autoprefixer tsx
```

2.2 docker-compose.yml

Create at repo root and start it first.

```
services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_USER: dealflow
      POSTGRES_PASSWORD: dealflow
      POSTGRES_DB: dealflow
    ports: ["5432:5432"]
    volumes: ["pgdata:/var/lib/postgresql/data"]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U dealflow"]
      interval: 5s
      retries: 10

  redis:
    image: redis:7
    ports: ["6379:6379"]

volumes:
  pgdata:
```

2.3 .env.local

```
DATABASE_URL="postgresql://dealflow:dealflow@localhost:5432/dealflow"
REDIS_URL="redis://localhost:6379"
JWT_SECRET="replace-with-32-byte-random-string"
PORTAL_TOKEN_SECRET="replace-with-different-32-byte-string"
ACCESS_TOKEN_TTL="15m"
REFRESH_TOKEN_TTL_DAYS="7"
```

---

3. Repository layout

```
dealflow360/
  docker-compose.yml
  prisma/
    schema.prisma
    seed.ts
  worker/
    index.ts                  # BullMQ worker, separate process
    jobs/
  src/
    middleware.ts             # Edge: JWT gate only
    app/
      (auth)/login, /signup
      (app)/                  # authenticated shell
        quotes/, quotes/[id]/
        approvals/
        dashboard/
        admin/
      portal/[token]/         # customer-facing, no session
      api/
        auth/{signup,login,refresh,logout}/route.ts
        quotes/route.ts
        quotes/[id]/{route,lines,submit,approve,reject,allocate,confirm}/
        products/route.ts
        pricing/preview/route.ts
        upsell/route.ts
        portal/[token]/{route,counter-offer}/
        dashboard/{metrics,anomalies}/
    lib/
      db.ts                   # Prisma singleton
      redis.ts                # ioredis singleton
      auth/{password,jwt,session,rbac}.ts
      cache.ts                # cache-aside helpers
      ratelimit.ts            # token bucket in Redis
      audit.ts                # append-only writer
      contracts/              # Zod schemas, shared
      services/
        pricing.ts  discount.ts  approval.ts
        allocation.ts  billing.ts  subscription.ts
        upsell.ts  anomaly.ts  portal.ts
    components/
```

---

4. Data model

Write `prisma/schema.prisma` first, in full, before any feature code. Enums and money precision are mandatory.

4.1 Enums

```
enum Role            { SALES_REP SALES_MANAGER FINANCE ADMIN }
enum CustomerTier    { BRONZE SILVER GOLD PLATINUM }
enum QuoteStatus     { DRAFT PENDING_APPROVAL APPROVED REJECTED NEGOTIATING CONFIRMED }
enum ApprovalStatus  { PENDING APPROVED REJECTED }
enum AllocationStatus{ RESERVED BACKORDERED SHIPPED }
enum BillingType     { ONE_TIME RECURRING }
enum AnomalyKind     { DEEP_DISCOUNT NEGATIVE_MARGIN EXCESSIVE_REAPPROVAL }
```

4.2 Models

Required models and their essential fields. The agent may add timestamps and indexes freely but must not rename these.

User — id, email (unique), passwordHash, name, role: Role, approvalLimitPct: Decimal(5,2), createdAt

RefreshToken — id, userId, tokenHash (unique), expiresAt, revokedAt?

Customer — id, name, email, tier: CustomerTier

Product — id, sku (unique), name, category, listPrice: Decimal(12,2), unitCost: Decimal(12,2), billingType: BillingType

PriceList / PriceListItem — priceList: id, name, currency; item: priceListId, productId, price: Decimal(12,2)

DiscountRule — id, customerTier, productCategory, maxAutoApprovePct: Decimal(5,2), requiredRole: Role

Quote — id, quoteNumber (unique), customerId, ownerId, status: QuoteStatus, subtotal, discountTotal, taxTotal, grandTotal, blendedDiscountPct: Decimal(5,2), marginPct: Decimal(5,2), reapprovalCount: Int @default(0), createdAt, updatedAt

QuoteLine — id, quoteId, productId, qty: Int, unitPrice: Decimal(12,2), discountPct: Decimal(5,2), lineTotal: Decimal(12,2),

billingType, subscriptionMonths: Int?

Approval — id, quoteId, approverId?, requiredRole: Role, status: ApprovalStatus, reason?, decidedAt?, cycle: Int

Warehouse — id, code (unique), name, region

Stock — id, warehouseId, productId, qtyOnHand: Int, version: Int @default(0), `@@unique([warehouseId, productId])`

Allocation — id, quoteLineId, warehouseId?, qty: Int, status: AllocationStatus

Invoice — id, quoteId, invoiceNumber (unique), amount: Decimal(12,2), issuedAt, dueAt

SubscriptionLine — id, quoteLineId, monthlyAmount: Decimal(12,2), startDate, months, proratedFirstAmount: Decimal(12,2)

UpsellRule — id, triggerProductId?, triggerCategory?, recommendedProductId, kind (`UPSELL` | `CROSS_SELL`), priority: Int

PortalToken — id, quoteId, tokenHash (unique), expiresAt, revokedAt?

AuditLog — id, entityType, entityId, action, actorId?, beforeJson: Json?, afterJson: Json?, createdAt. **Append-only. Never update or delete a row in this table.**

Anomaly — id, quoteId, kind: AnomalyKind, detail, detectedAt

4.3 Required indexes

```
@@index([status])                       // Quote, for approver queue
@@index([quoteId])                      // QuoteLine, Approval, Allocation
@@index([entityType, entityId])         // AuditLog
```

---

5. API contract

All responses are JSON. Errors use `{ "error": { "code": string, "message": string } }` with appropriate HTTP status. Every handler validates its body with a Zod schema from `lib/contracts/`.

Method	Path	Role	Purpose
POST	`/api/auth/signup`	public	Create user
POST	`/api/auth/login`	public	Set access + refresh cookies
POST	`/api/auth/refresh`	cookie	Rotate access token
POST	`/api/auth/logout`	cookie	Revoke refresh token
GET	`/api/products`	any	List with price list resolution (cached)
POST	`/api/quotes`	REP	Create draft
GET	`/api/quotes/:id`	owner/mgr	Full quote with lines, allocations, approvals
POST	`/api/quotes/:id/lines`	REP	Add line, recompute totals
POST	`/api/pricing/preview`	REP	Recompute totals without persisting
GET	`/api/upsell?quoteId=`	REP	Recommendations for current lines
POST	`/api/quotes/:id/submit`	REP	DRAFT → PENDING_APPROVAL or APPROVED
POST	`/api/quotes/:id/approve`	MANAGER	Approve current approval cycle
POST	`/api/quotes/:id/reject`	MANAGER	Reject with reason
POST	`/api/quotes/:id/allocate`	FINANCE	Split stock across warehouses
POST	`/api/quotes/:id/confirm`	FINANCE	Generate invoice + subscriptions
GET	`/api/portal/:token`	portal token	Customer view of one quote
POST	`/api/portal/:token/counter-offer`	portal token	Request lower price
GET	`/api/dashboard/metrics`	MANAGER	Aggregates
GET	`/api/dashboard/anomalies`	MANAGER	Open anomalies

---

6. Business logic specifications

6.1 Pricing (`lib/services/pricing.ts`)

For each line: `lineTotal = unitPrice * qty * (1 - discountPct/100)`, rounded to 2 dp using `Prisma.Decimal`.

Quote-level:

- `subtotal` = sum of `unitPrice * qty`
- `discountTotal` = `subtotal - sum(lineTotal)`
- `blendedDiscountPct` = `discountTotal / subtotal * 100` — this is the number approval routing uses, **not** the max line discount
- `marginPct` = `(sum(lineTotal) - sum(unitCost * qty)) / sum(lineTotal) * 100`

6.2 Discount governance and approval routing (`discount.ts`, `approval.ts`)

On submit:

1. For each line, look up the `DiscountRule` matching the customer's tier and the product's category.
2. If every line's `discountPct` is at or below its rule's `maxAutoApprovePct` **and** `blendedDiscountPct` is at or below the highest applicable `maxAutoApprovePct`, set status `APPROVED` immediately.
3. Otherwise take the strictest matched rule's `requiredRole`, create an `Approval` row with `cycle = quote.reapprovalCount + 1`, and set status `PENDING_APPROVAL`.
4. Write an `AuditLog` row for the transition.

On approve: verify the approver's `approvalLimitPct` from the **database**, not from JWT claims, is at least `blendedDiscountPct`. Reject with 403 if not. Set quote `APPROVED`, enqueue a notification job, write audit.

6.3 Warehouse allocation (`allocation.ts`) — concurrency critical

This is the only place in the project where two users can corrupt data. Implement exactly as follows.

```
        await prisma.$transaction(async (tx) => {
          const rows = await tx.$queryRaw`
            SELECT id, "warehouseId", "qtyOnHand"
            FROM "Stock"
            WHERE "productId" = ${productId} AND "qtyOnHand" > 0
            ORDER BY "qtyOnHand" DESC
            FOR UPDATE
          `;
          // greedily consume from largest stock first
          // create Allocation rows with status RESERVED
          // decrement qtyOnHand
          // remainder -> one Allocation with warehouseId null, status BACKORDERED
        }, { isolationLevel: "Serializable" });
```

Verify with a script that fires two concurrent allocations for the last remaining unit: exactly one must succeed with `RESERVED`, the other must receive `BACKORDERED`. Stock must never go negative.

6.4 Subscription proration (`subscription.ts`)

For a `RECURRING` line starting mid-month:

```
daysRemaining     = daysInMonth - startDay + 1
proratedFirst     = monthlyAmount * daysRemaining / daysInMonth
```

Round to 2 dp. Store on `SubscriptionLine`. Invoice amount = one-time line totals + prorated first period.

6.5 Customer negotiation (`portal.ts`)

A counter-offer sets the requested discount on the quote, increments `reapprovalCount`, sets status `NEGOTIATING`, then immediately re-runs section 6.2 routing, producing a new `Approval` row with the incremented `cycle`. Old approval rows are never modified — the cycle history is the audit trail.

6.6 Anomaly detection (`anomaly.ts`)

Three deterministic rules only. No machine learning.

- `DEEP_DISCOUNT` — `blendedDiscountPct` exceeds the tier ceiling by more than 10 points
- `NEGATIVE_MARGIN` — any line where `lineTotal < unitCost * qty`
- `EXCESSIVE_REAPPROVAL` — `reapprovalCount >= 3`

Run on every quote state change, inside the BullMQ worker, not in the request path.

6.7 Caching (`cache.ts`)

Cache-aside on Redis with 300 s TTL for: resolved price lists (`price:{priceListId}`), product catalog (`catalog:all`), upsell rules (`upsell:rules`). Invalidate by deleting the key on any admin write. Nothing else is cached.

6.8 Rate limiting (`ratelimit.ts`)

Token bucket in Redis, applied **only** to `/api/portal/*`: 30 requests per minute per IP, and 100 per hour per portal token. Return HTTP 429 with `Retry-After`. Internal authenticated routes are not rate limited.

6.9 Authentication (`lib/auth/*`)

- Hash with `argon2id`, default parameters.
- Access token: JWT via `jose`, 15 minute expiry, claims `{ sub, role }`. Stored in an httpOnly, SameSite=Lax cookie.
- Refresh token: 32 random bytes, SHA-256 hashed into `RefreshToken`, 7 day expiry, httpOnly cookie. Rotate on every refresh and revoke the old row.
- Portal tokens are **not** user sessions. Generate random bytes, store the hash in `PortalToken`, put the raw value in the shareable URL. Every portal query must be scoped to `portalToken.quoteId` — never trust a quote id from the request body.

---

7. Build phases

Phase 0 — Foundation (target: 90 minutes)

1. `pnpm create next-app` with TypeScript, Tailwind, App Router, `src/` directory.
2. Add dependencies from 2.1, write `docker-compose.yml`, `.env.local`.
3. `docker compose up -d`, wait for healthcheck.
4. Write `prisma/schema.prisma` in full per section 4.
5. `pnpm prisma migrate dev --name init`
6. Write `lib/db.ts` and `lib/redis.ts` singletons.
7. Write all Zod schemas in `lib/contracts/` from section 5.

Verify: `pnpm prisma studio` opens and shows every table.

Phase 1 — Seed data

Write `prisma/seed.ts` producing, deterministically:

- 4 users, one per role, password `password123`
- 5 customers spanning all four tiers
- 20 products across at least 3 categories, mixed `ONE_TIME` and `RECURRING`
- 1 default price list covering all products
- Discount rules for every tier × category combination
- 3 warehouses with **deliberately uneven stock**, including:
  - one product with exactly **1 unit** in exactly one warehouse (concurrency demo)
  - one product whose stock forces a **two-warehouse split plus a backorder** (fulfillment demo)
- 6 upsell rules
- 1 quote pre-seeded in `PENDING_APPROVAL` (so the approver queue demos without building a quote live)

Add scripts: `"db:reset": "prisma migrate reset --force && prisma db seed"`.

Verify: `pnpm db:reset` returns to a known-good state in under 15 seconds.

Phase 2 — Auth and shell

Auth routes, middleware JWT gate, login/signup pages, authenticated layout with role-gated navigation, audit writer.

Verify: log in as each of the four roles; each sees only its permitted navigation; a REP hitting `/api/quotes/:id/approve` receives 403.

Phase 3 — Quote workspace and pricing

Product catalog with cache, quote creation, add/remove lines, live pricing preview, blended discount and margin display, upsell panel.

Verify: adding a line recomputes totals; second identical catalog request is served from Redis.

Phase 4 — Discount approval

Routing per 6.2, approver queue page, approve/reject with reason, full audit trail on the quote detail page.

Verify: a 5% discount on a PLATINUM customer auto-approves; a 40% discount routes to SALES_MANAGER and appears in that manager's queue.

Phase 5 — Fulfillment

Allocation per 6.3, warehouse split display, backorder rows, concurrency test script.

Verify: the two-concurrent-request script produces exactly one RESERVED and one BACKORDERED, and stock never goes negative.

Checkpoint — the spine must be demoable here. Log in → build quote → deep discount → routes to manager → approve → allocate splits across two warehouses with a backorder. Record a screen capture now as insurance.

Phase 6 — Billing and subscriptions

Invoice generation on confirm, subscription lines with proration, mixed one-time and recurring on a single quote.

Verify: a quote with one hardware line and one monthly line starting on the 20th produces an invoice with a correctly prorated first period.

Phase 7 — Customer portal

Portal token issuance and share link, read-only quote view, counter-offer form, rate limiting.

Verify: the counter-offer flips status to NEGOTIATING, creates a cycle-2 Approval row, and reappears in the manager queue. A 40-request burst returns 429.

Phase 8 — Worker, dashboard, anomalies

BullMQ worker process, anomaly scan on state change, dashboard with pipeline value by status, approval turnaround, discount distribution, and an open-anomaly list.

Verify: `pnpm worker` runs alongside `pnpm dev`; a deep-discount quote produces a DEEP_DISCOUNT anomaly within seconds.

Phase 9 — Freeze

No new features. `pnpm db:reset`, run the full demo three times, fix only breakage, write README with setup steps and demo script.

---

8. Team split (three developers)

Boundaries follow the schema so no two people write the same tables.

Developer A — Pricing, discounts, approvals. Sections 6.1, 6.2, 6.7 partially, and Phases 3 and 4. Owns `pricing.ts`,

`discount.ts`, `approval.ts`, `upsell.ts`. This is the deepest logic and the centrepiece of the demo.

Developer B — Fulfillment, billing, infrastructure. Sections 6.3, 6.4, 6.6, and Phases 1, 5, 6, and the worker in 8. Owns `allocation.ts`, `billing.ts`, `subscription.ts`, `anomaly.ts`, `docker-compose.yml`, `seed.ts`. The locking transaction is the highest-risk item in the project.

Developer C — Auth, portal, UI shell, dashboard. Sections 6.5, 6.8, 6.9, and Phases 2, 7, and the dashboard in 8. Owns `lib/auth/*`, `ratelimit.ts`, `audit.ts`, `portal.ts`, the app shell, quote builder UI, approver queue, and dashboard.

C is the integration point and will be busiest near the end. Whoever finishes first joins C.

8.1 Working agreement

- Nobody edits `prisma/schema.prisma` without all three agreeing. A silent schema change breaks the other two.
- Merge to `main` every two hours, not at phase boundaries. Long-lived branches of AI-generated code produce conflicts that cost more than the code did.
- If a module needs something from another module, add the function signature to `lib/services/` as a stub returning mock data, and let the owner fill it in. Do not reach into their tables.

8.2 Agent orchestration notes

Run one agent per developer against their own module. Keep spare agent slots for test generation and for fixing already-merged code, not for a fourth feature.

Prompt agents with the contract, never with prose. Effective form:

> *Implement `POST /api/quotes/:id/allocate` per section 5 and the algorithm in section 6.3 of IMPLEMENTATION.md, using the `Stock` and `Allocation` models in prisma/schema.prisma. Use `export const runtime = "nodejs"`.*

An agent given the schema and the endpoint contract produces mergeable code. An agent given a paragraph of English invents its own field names.

---

9. Scope control

If you fall behind, cut in this order. Do not cut upward.

1. Quote PDF export
2. Recurring billing beyond a single plan type
3. Dashboard charts beyond two
4. Admin configuration UI — seed the data instead
5. Upsell recommendations

Never cut: the quote → approval → allocation → invoice spine, the locking transaction, the audit trail, or the portal counter-offer. Those four are what the specification is actually asking for.